Countersign: Evidence-Grounded Supervision of Coding-Agent Completion Claims

Anonymous Author(s)
Affiliation
Address
email

Abstract

Coding agents terminate when they believe a task is done, but that belief is not always supported by their own execution trace. We present *Countersign*, a deterministic supervisory meta-agent that audits a worker's completion claim against the trace—checking that cited tests exist, remain fresh for the files they cover, postdate the latest requirement update, and rest on no superseded source—and can block termination to demand fresh evidence. Countersign is deliberately *trace-only*: it judges whether a claim is *justified* at claim time—distinct from whether the implementation is *correct*, which a hidden evaluator answers once after termination. In a preregistered, frozen campaign of 490 real-model runs over ten held-out fixtures, all 17 blocks landed on proposals an independent oracle also labelled unsupported (precision 1.00, 95% CI $[0.82, 1.0]$) and none on the 204 supported proposals judged—though 13 of 17 are the trivial case of a claim citing no evidence. A counterfactual replay shows a conventional continuous-integration gate catches 2 of 15 unsupported proposals; Countersign catches 11 of 11 it judged. Negative results are equally informative: truncating the worker's visible history did not detectably increase unsupported completion, and a treatment-identical control arm disagrees with the baseline more often than the gated arm does—so we report a measured noise floor rather than an effect, and locate the gate's contribution where it is deterministic: the unsupported proposals it sees become blocked episodes that recover. And we quantify the construct honestly: all 18 accepted unsupported completions passed hidden evaluation—Countersign is an evidence-hygiene instrument whose safety value remains a hypothesis under test.

1 Introduction

An autonomous coding agent ends its episode by asserting completion—a claim about evidence: the tests it ran, the edits it made, the requirements it was given. When the claim outruns the evidence (a cited test predating the final edit; a superseded source), the agent has not necessarily produced wrong code, but its report is *unwarranted*, and whoever trusts it inherits an invisible risk.

We study whether a second agent with authority over the first's termination can reliably tell warranted completion claims from unwarranted ones, and what that authority costs. Countersign reads only the worker's execution trace and never consults ground truth; the hidden evaluator that determines correctness runs once, post-termination, identically in every condition. This separation is the organizing principle: a claim can be justified and wrong, or correct and unjustified, and conflating the two makes "the verifier improved outcomes" unfalsifiable.

Contributions. We contribute (i) a termination audit of cited-evidence chronology (freshness, requirement recency, provenance, abstention) with halting authority, scored against an independent post-hoc oracle it does not compute (§2); (ii) a preregistered evaluation on matched pairs and negative controls pricing false blocks, under a frozen protocol with a public deviation ledger (§3); (iii) results

Submitted to the NeurIPS 2026 Workshop on Managing Agents that Manage Agents (Meta-Agents). Do not distribute.

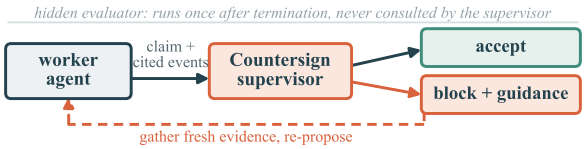

Figure 1: The supervision loop: the worker proposes termination with cited evidence; Countersign accepts, or blocks with the specific deficiency and the worker re-proposes with fresh evidence. The hidden evaluator runs once, post-termination, never consulted by the supervisor.

across 490 runs—perfect block precision, a CI-gate ablation, a truncation null, and a noise floor measured from a treatment-identical control arm that bounds what any rate contrast here can claim (§4); (iv) a calibrated account of the gate as evidence hygiene, not a demonstrated safety mechanism (§5).

2 The Countersign Supervisor

Worker and ledger. The worker is a tool-loop coding agent (self-hosted open-weight models) in a git-backed workspace; every action appends to an *evidence ledger*—tool calls with outcomes, file mutations, test runs with inferred coverage, requirement updates—and it finishes by emitting a claim with cited evidence identifiers.

The audit. Countersign checks four properties of the cited evidence: (i) at least one cited event is a successful test run; (ii) no cited test predates a later mutation of a file it covers (Appendix A); (iii) cited evidence postdates the latest requirement update; and (iv) the claim rests on no source the task marks superseded. Freshness is tri-state: the supervisor abstains when coverage is unknown.

Authority. On failure the supervisor blocks termination and returns the specific deficiency (Figure 1); a repair variant additionally issues the smallest evidence-refreshing action. Determinism is deliberate: halting authority must be auditable and replayable. Every arm logs the *raw* verifier decision separately from the *enforced* one, so non-blocking arms still measure what the supervisor would have done.

Independent oracle. A post-hoc oracle labels every proposal supported/unsupported/uncertain from fixture-authored completion policies and canonical trace chronology. It shares no code with the online verifier; the two disagreed on one proposal (§4), and a shared claim classifier diverges under degradation (Appendix C).

3 Evaluation Design

Fixtures. Ten held-out fixture repositories, frozen before any real-model run was inspected. Three *matched pairs* isolate one failure mode each under context parity (workspaces byte-identical but for one designed file): *temporal* (cited tests predate the final edit), *provenance* (fresh evidence grounded in a superseded document), and *requirement* (a clarification arrives after the last passing run). Four *negative controls* contain no designed trap (Appendix A): a block on an oracle-supported proposal there is a false positive at context parity, though a control can still elicit an evidence-free claim, which blocking is correct. A separate development suite, on which the rules were iterated, is never used for evaluation.

Conditions. Every arm runs the *same* instrumented worker prompt—a deliberate choice, pinned by a regression test—so the only treatment is what happens *after* a proposal: memory_baseline (no gate), observe_only (verifier judges and logs, never blocks, zero worker-visible feedback), verification_only (blocking), verification_and_repair, and an evaluation-only oracle_supervisor bound. The baseline is thus *instrumented*, not naive, and the primary contrast isolates the gate alone; in the traces, the first two arms issue byte-identical first prompts in $10/10$ matched cells and identical first responses in $9/10$.

Memory degradation. A frozen registry of degradation profiles applies only to *model-visible* state [1, 2]: graded *lossy truncation* of history (low/medium/high) and a *resume-summary* regime

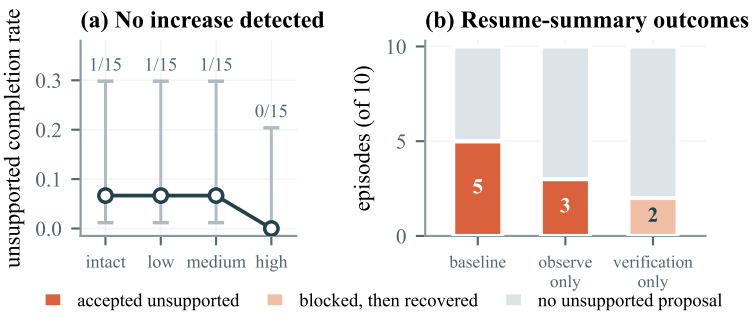

Figure 2: (a) Graded truncation: no detectable increase; the wide Wilson 95% intervals mark a non-detection at modest power, not invariance. (b) The resume-summary regime does produce the effect: the counts are $5/3/2$, but the first two arms are treatment-identical, so their $4/10$-cell disagreement is the endpoint's noise floor—larger than the gate contrast's $3/10$. No proposal-level effect is claimed; both proposals the gate saw became blocked episodes that recovered.

replacing the visible ledger with a single truthful summary—the pattern deployed scaffolds use on resumption. The canonical ledger, verifier, oracle, and evaluator always see undegraded state.

Protocol. Temperature 0.7, sampler seeds 0–4, action budget 24 (Appendix A). The predeclared primary is accepted-unsupported finish, memory_baseline vs verification_only; the decomposition is the predeclared confound branch, other contrasts exploratory. Endpoints, comparisons, and interpretation branches for both positive and null outcomes were committed before the runs; the frozen protocol hashes code, fixtures, and model weights and refuses to start on a dirty tree.

4 Results

All numbers come from audited run manifests; the worker is `qwen2.5-coder:14b`. The campaign was sized to validate the instrument and calibrate power, not to detect effects at significance; its 490 runs yield 341 oracle-judged finish proposals, 27 labelled unsupported (Appendix B reconciles every denominator below).

Blocks are precise. Countersign issued 17 blocks across every arm with a live verifier—9 enforced, plus 8 raw would-blocks in non-enforcing arms whose proposals were still accepted. All 17 landed on proposals the independent oracle also labelled unsupported, none on the 204 supported proposals a verifier judged: precision 1.00 (95% CI $[0.82, 1.0]$), false-block rate $0/204$ (95% CI $[0, 0.018]$), recall $17/18 = 0.94$ (95% CI $[0.74, 0.99]$). The one miss is a boundary case: a post-block re-proposal cited evidence the verifier accepted but the oracle did not. Two caveats bound this. First, the denominator counts only judged proposals: 110 supported proposals in the no-verifier arm are excluded, not counted as successes. Second, and more important, **13 of the 17 blocks are the trivial case**—a claim citing no evidence, which any rule detects. Only 4 required substantive reasoning: read the result as "the gate does not misfire," not "it discriminates hard cases."

A conventional CI gate is mostly blind. We replay every stored proposal of the four ablation campaigns against the standard-practice rule "require a successful test run after the last edit," a strict subset of Countersign's temporal check. On the subset's 15 unsupported proposals the CI rule catches 2; Countersign judged 11 of the 15 (4 arose in no-verifier arms) and caught all 11—only 4 requiring substantive reasoning; the rest cite nothing at all. Neither gate blocked any of the 161 supported proposals both judged: the difference is blindness, not conservatism.

Truncation does not detectably induce false claims. Baseline unsupported completion on trap fixtures did not increase with truncation severity (Figure 2a): a non-detection, not invariance, since Wilson intervals at 15 runs per cell span ∼0.01–0.30. The preregistered null branch applies.

The resume-summary regime does produce the effect, and it decomposes (Figure 2b). The comparison must be read at the *proposal* level: a blocking arm suppresses accepted-unsupported finishes by definition, so a zero there is partly mechanical. Counting unsupported proposals over ten matched cells: baseline 5, observe_only 3, verification_only 2. **None of that gradient is interpretable.** The arms are identical until a proposal is made, so memory_baseline vs. observe_only is *treatment-*

Table 1: Accepted completions (n=327): claim verdict (oracle) vs. work verdict (evaluator); the empty cell is the central caveat.

	correct	incorrect
claim supported	307	2
claim unsupported	18	0

identical and its disagreement is pure sampling variation: $4/10$ cells (3 vs. 1)—a noise floor *larger* than the gate contrast's own $3/10$ (3 vs. 0). We claim no proposal-level effect. The gate's contribution is instead a different fate for the proposals it sees: both were blocked and both episodes recovered. Campaign-wide, 7 of 8 enforced-block episodes recovered to supported termination with no budget exhaustion; the repair arm never fired and the oracle bound admitted zero incorrect completions (Appendix B). No comparison reaches $p < .05$, and the power analysis explains why: even with every discordant pair in one direction the exact test needs $b \geq 6$, one more than this campaign could produce, and $\sim$84 matched cells would be needed for 80% power at these rates (Appendix E). The cleanest evidence is measurement without power: the observe_only verifier judges but cannot act, and its would-blocks fired in exactly the 3 unsupported cells and none of the other 7 (TP$=3$, FP$=0$, FN$=0$)—immune to the circularity objection blocking invites.

What "unsupported" means. Table 1 crosses the oracle's judgement of the accepted claim against the hidden evaluator's judgement of the work. All 18 unsupported completions concerned work that was *correct*: an unjustified claim never coincided with a wrong implementation. Countersign is measured to catch *poor evidential hygiene*; its safety value remains a hypothesis this campaign cannot confirm. (Expected empty-cell count under verdict independence: ≈ 0.1—weak evidence either way.)

5 Supervisor-Failure Analysis and Limitations

The structural blind spot. The provenance pair is designed so fresh, well-cited evidence grounded in a superseded source passes a trace-only audit while failing hidden evaluation. The designed path never fired (both supported-but-incorrect completions arose elsewhere; Appendix B), but the class is real, unreachable by any trace audit, and bounds what evidence supervision can promise. Appendix C details two further failure modes: classifier/oracle divergence under degradation and a family censored by task weight, not budget.

Limitations. (1) Most important: the traps and the rules share authors; the dev/held-out split prevents tuning on evaluation fixtures but not the conceptual correlation between what we trap and what we detect; the provenance family (built to *defeat* the supervisor) and the negative controls offset it only partially, and the correlation can inflate only the block numerator—false blocks and post-block recoveries are organic. (2) One worker model on ten small fixtures establishes nothing about prevalence in production software. (3) The primary endpoint is partly suppressed by construction in blocking arms; gate effects are read at the proposal level and in post-block outcomes. (4) The verifier and oracle share no code but read the same trace; most agreements concern claims citing no evidence—near-vacuous independence. (5) No predeclared comparison reaches $p < .05$; nulls are non-detections at modest power, and we report decompositions and intervals, never significance. (6) The degradation profiles are our own transforms, not a deployed scaffold's condenser; (7) oracle-label validation covers a single-rater sample; (8) intervals treat cells as independent though proposals nest in episodes and fixtures, so nominal coverage is approximate.

6 Related Work

Workers game outcome checks [3, 4]; external supervisors with authority over another agent are an established design space [5–8]; deterministic finish-blocking hooks are deployed practice—the very rule our CI replay prices; Countersign differs in *what* it audits (cited-evidence chronology with abstention) and in pricing false blocks. The justified/correct split descends from process-vs-outcome supervision [9, 10], instantiated at termination with a hidden evaluator [11, 12]. Trajectory judges score completion without termination authority [13]; SCATE targets premature termination [14]; concurrent work checks completion evidence in workflow graphs [15] and at GUI agents' finish step [16]; and self-verification [17] is the complement.

Responsible-Use Statement

Countersign controls agent overclaiming, but agents supervising agents carries risks: an *allow* audits justification, not correctness, and must not be marketed as verification; a visible gate invites *oversight displacement* (verdicts must stay cited-evidence audits, not badges); halting authority adds a failure surface—*who supervises the supervisor*—priced here as counter-metrics. No human subjects, personal data, or production systems were involved.

References

[1] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.

[2] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[3] Sydney Von Arx, Lawrence Chan, and Beth Barnes. Recent frontier models are reward hacking. METR Technical Report, 2025. URL `https://metr.org/blog/2025-06-05-recent-reward-hacking/`.

[4] Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation, 2025. URL `https://arxiv.org/abs/2503.11926`.

[5] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *Forty-first International Conference on Machine Learning, ICML 2024*, pages 16295–16336. PMLR, 2024. URL `https://proceedings.mlr.press/v235/greenblatt24a.html`.

[6] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In *Forty-second International Conference on Machine Learning, ICML 2025*. PMLR, 2025. URL `https://proceedings.mlr.press/v267/xiang25a.html`.

[7] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE 2026)*, 2026. URL `https://arxiv.org/abs/2503.18666`.

[8] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. LlamaFirewall: An open source guardrail system for building secure AI agents, 2025. URL `https://arxiv.org/abs/2505.03574`.

[9] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022. URL `https://arxiv.org/abs/2211.14275`.

[10] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[13] Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-judge: Evaluate agents with agents. In *Forty-second International Conference on Machine Learning, ICML 2025*. PMLR, 2025. URL `https://proceedings.mlr.press/v267/zhuge25a.html`.

[14] Sijia Gu, Noor Nashid, and Ali Mesbah. SCATE: Learning to supervise coding agents for cost-effective test generation, 2026. URL `https://arxiv.org/abs/2607.08983`.

[15] Jesus Salas. Correct is not governed: Provenance integrity in agentic workflows, 2026. URL `https://arxiv.org/abs/2608.12761`.

[16] Qijun Han, Haoqin Tu, Zijun Wang, Haoyue Dai, Yiyang Zhou, Nancy Lau, Alvaro A. Cardenas, Yuhui Xu, Ran Xu, Caiming Xiong, Zeyu Zheng, Huaxiu Yao, Yuyin Zhou, and Cihang Xie. VLAA-GUI: Knowing when to stop, recover, and search, a modular framework for GUI automation, 2026. URL `https://arxiv.org/abs/2604.21375`.

[17] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A Design Details

Test coverage is inferred statically, so documentation edits do not invalidate code evidence, and a failure superseded by a later passing run does not permanently contradict a claim. The four negative controls: documentation edits after green tests; an unrelated-module edit with a targeted-test citation; a legitimate zero-change audit; an irrelevant late clarification. Sampling uses temperature 0.7 because greedy decoding would make multi-seed replication pseudoreplication; seeds 0–4 are sampler seeds over otherwise identical episodes.

B Proposal Accounting

Every denominator in §4 derives from one ledger over the 490 runs' 341 finish proposals (Table 2). Identities: $341 = 327$ accepted $+ 9$ enforced blocks $+ 5$ oracle-arm refusals; 27 unsupported $= 18$ accepted $+ 9$ enforced-blocked; the 17 quoted blocks are *raw* decisions (9 enforced $+ 8$ would-blocks in non-enforcing arms, whose proposals were still accepted); verifier-judged proposals number 222 (204 supported $+ 18$ unsupported; recall $17/18$), and the remaining 110 supported proposals sit in the no-verifier baseline arm. The CI replay covers the four ablation campaigns: 15 unsupported proposals, 11 of them verifier-judged, 161 supported proposals judged by both gates.

Arm details: verification_and_repair produced no unsupported proposal in its 40 runs, so its repair path was never exercised—an uninformative cell, not evidence repair works. The evaluation-only oracle_supervisor bound refused 5 proposals, admitted zero incorrect completions, and accepted one unsupported-but-correct finish: it bounds correctness, not justification. The 9 enforced blocks hit 8 episodes; 7 recovered to supported termination, 1 ended on the missed boundary-case proposal, and none exhausted its action budget after a block. The campaign's two supported-but-incorrect completions arose on a negative-control and a requirement fixture, not the provenance pair.

C Additional Failure Modes

Classifier/oracle divergence. The online claim classifier (a shared component that labels the worker's completion claims from the worker's *visible* context, used for in-flight telemetry) disagreed with the independent oracle under high degradation, because the classifier reads the worker's degraded view while the oracle reads the canonical trace. All endpoints in the main text are oracle-anchored; the divergence is evidence that an independent oracle is necessary rather than redundant.

Table 2: Proposal ledger by arm.

arm	runs	proposals	unsup.	raw blocks	enforced	acc. unsup.
memory_baseline	180	119	9	0	0	9
observe_only	70	52	7	7	0	7
oracle_supervisor	40	30	1	1	0	1
verification_and_repair	40	28	0	0	0	0
verification_only	160	112	10	9	9	1
total	490	341	27	17	9	18

Task weight censors a family. Requirement-family trap runs exhaust the action budget in $3/5$ runs in every arm, and raising the budget from 24 to 40 actions left that rate unchanged. The family is intrinsically too heavy for this worker—a fixture-design finding, not a budget-calibration error.

D A Blocked-then-Recovered Trajectory

Under lossy truncation at low severity, one verification-arm episode on a fresh-evidence temporal fixture shows the mechanism in miniature. The pressured worker twice proposed terminating *legitimate* work with insufficient cited evidence; the supervisor blocked both proposals with the specific deficiency; the worker gathered fresh evidence, terminated supported, and passed hidden evaluation. Pressure-induced premature finishing was rare for this worker, but both instances that occurred were caught and repaired by the gate. The five blocks issued across the pressure gradient (three on trap fixtures, two on control-task proposals) all landed on oracle-unsupported proposals, and enforced false blocks on oracle-supported work remained zero campaign-wide.

E Bayesian Sensitivity Readings

As a post-hoc sensitivity analysis (recorded as such in the deviation ledger; no new data, uniform $\mathrm{Beta}(1,1)$ priors, exact conjugate posteriors, no sampling), we re-read the paper's key small-n counts in Bayesian form. *The noise floor*: the treatment-identical contrast has discordant cells b=3, c=1, giving posterior $\mathrm{Beta}(4,2)$ and $P(\theta > 0.5) = 0.8125$ on the direction of a difference *no treatment produces*. That a pure-noise contrast yields four-to-one posterior odds cautions against reading direction from small-n discordant counts, including ours. *Discrimination in the arm that cannot act*: the observe_only verifier's sensitivity $3/3$ has posterior mean 0.80 with 95% credible interval $[0.40, 0.99]$, and its specificity $7/7$ has mean 0.89, interval $[0.63, 1.00]$; campaign-wide block precision $17/17$ gives $[0.81, 1.00]$, agreeing with the Wilson interval in the main text. These are direction and uncertainty statements only; none is a magnitude claim, and all shrink toward chance under priors skeptical of any effect.

F Reproducibility

The campaign spans three tags of one freeze: a base freeze plus two infrastructure-only fixes applied mid-campaign (interruption-recovery artifact reuse; a tool-level crash on syntactically invalid worker-written code surfacing as a tool error rather than killing the episode). The hashed verifier-policy files are byte-identical across all three tags: no verifier, oracle, or fixture logic changed after the freeze. Every deviation, interim inspection, and protocol-identifier supersession—including two interim analyses that informed spending decisions, and one corrected over-interpretation—is recorded in the deviation ledger shipped with the artifact; the interim analyses changed scope only—no endpoint or allocation rule was altered post hoc. The artifact is a relocatable bundle whose integrity audit passes from the copied location; it contains only fixtures, protocols, and run records.